

LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK
EKSPANSI PRODUK KEUANGAN (INHERITANCE)



Disusun Oleh:

Gina Ramadhani

NIM: 2511533014

Kelas: B

Dosen Pengampu: Nurfiah, S.ST, M.Kom

Asisten Praktikum: Siti Fadhilah Rahmi

DAPERTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

DAFTAR ISI

DAFTAR ISI	i
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
A. Langkah Kerja Praktikum	2
B. Tugas/Latihan Praktikum	5
C. Link repository GitHub	17
BAB III PENUTUP	18
3.1 Kesimpulan.....	18
DAFTAR PUSTAKA.....	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan sistem perbankan membutuhkan program yang mampu mengelola berbagai jenis produk rekening dengan karakteristik yang berbeda. Dalam pemrograman berorientasi objek, konsep Inheritance (pewarisan) dapat digunakan untuk membuat class turunan dari sebuah class utama sehingga kode menjadi lebih terstruktur dan mudah dikembangkan. Pada praktikum Ekspansi Produk Keuangan (Inheritance), class Rekening dikembangkan menjadi RekeningTabungan dan RekeningGiro dengan fitur khusus seperti suku bunga dan batas overdraft. Praktikum ini juga menerapkan konsep upcasting, downcasting, dan **instanceof** untuk mengelola berbagai jenis rekening dalam satu sistem perbankan.

1.2 Tujuan Praktikum

1. Memahami prinsip Code Reusability melalui Pewarisan (Inheritance).
2. Membedakan Superclass dan Subclass
3. Menggunakan keyword `extends` untuk mewariskan sifat, serta `super()` untuk memanggil konstruktor induk.
4. Menerapkan Upcasting dan Downcasting pada koleksi objek

1.3 Manfaat Praktikum

1. Memahami penerapan konsep Inheritance dalam pemrograman berorientasi objek.
2. Memahami penggunaan upcasting dan downcasting pada class induk dan class turunan.
3. Mengetahui penggunaan operator *instanceof* untuk memeriksa jenis objek.
4. Meningkatkan kemampuan dalam mengembangkan sistem perbankan yang terstruktur dan mudah dikembangkan.

BAB II

PEMBAHASAN

A. Langkah Kerja Praktikum

Pada praktikum ini, kita akan merefaktorkan Rekening.java menjadi kelas induk (Superclass), lalu membuat produk spesifik berupa Tabungan dan Giro.

a) Langkah 1: Memastikan Akses Modifier di Rekening.java

Buka kembali file Rekening.java. Pastikan atribut saldo dan riwayatTransaksi menggunakan hak akses protected, sementara pin dan nomorRekening tetap private.

```
6 // 1. Mengunci atribut dengan 'private'
7 private String nomorRekening;
8 private String namaPemilik;
9 private String pin; // Data Sensitif//
10
11 protected double saldo;
12 protected ArrayList<Transaksi> riwayatTransaksi;
```

Berikut penjelasan Langkah-langkah kerjanya:

1. private String nomorRekening;
Menyimpan **nomor rekening** nasabah. private berarti atribut ini hanya dapat diakses langsung dari dalam class Rekening.
2. private String namaPemilik;
Menyimpan **nama pemilik rekening**. Menggunakan private agar data tidak dapat diakses atau diubah secara langsung dari luar class.
3. private String pin;
Menyimpan **PIN rekening**. Dibuat private karena PIN merupakan **data sensitif** yang harus dilindungi dan tidak boleh diakses langsung dari class lain.
4. protected double saldo;
Menyimpan **saldo rekening** dalam tipe double. protected memungkinkan atribut ini diakses oleh class Rekening dan class turunannya (subclass).
5. protected ArrayList<Transaksi> riwayatTransaksi;

Menyimpan **daftar riwayat transaksi** menggunakan ArrayList. protected memungkinkan class turunan mengakses daftar transaksi tersebut.

b) Langkah 2: Membuat Subclass RekeningTabungan

Buat file baru bernama RekeningTabungan.java. Produk tabungan memiliki fitur tambahan yaitu Suku Bunga Bulanan.

```
1 package pekan4;
2
3 public class RekeningTabungan extends Rekening {
4
5     //Atribut spesifik yang hanya dimiliki oleh tabungan
6     private double sukuBunga;
7
8     // Konstruktor Subclass
9     public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {
10         // super() memanggil konstruktor kelas induk (Rekening). WAJIB berada di baris pertama!
11         super(nomor, nama, saldoAwal, pinAwal);
12         this.sukuBunga = sukuBunga;
13     }
14
15     public void tambahBungaAkhirBulan() {
16         // menghitung bunga
17         // mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan ?
18         double nominalBunga = saldo * (sukuBunga / 100);
19         saldo += nominalBunga;
20
21         // Mencatat riwayat transaksi
22         String idTrx = "TRX-B" + System.currentTimeMillis();
23         riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));
24
25         System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);
26     }
27 }
```

Berikut penjelasan Langkah-langkah kerjanya:

1. public class RekeningTabungan extends Rekening

Menunjukkan bahwa RekeningTabungan merupakan **class turunan dari class Rekening**, sehingga dapat menggunakan atribut dan method yang diwariskan dari class induk.

2. private double sukuBunga;

Menyimpan **besar suku bunga rekening tabungan**. private berarti atribut ini hanya dapat diakses langsung dari dalam class RekeningTabungan.

3. public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga)

Merupakan **constructor** yang digunakan untuk membuat objek RekeningTabungan dengan menerima nomor rekening, nama pemilik, saldo awal, PIN, dan suku bunga.

4. super(nomor, nama, saldoAwal, pinAwal);

Memanggil **constructor** dari **class induk Rekening** untuk mengisi data nomor rekening, nama, saldo awal, dan PIN. `super()` wajib berada pada baris pertama constructor subclass.

5. `this.sukuBunga = sukuBunga;`

Mengisi atribut `sukuBunga` dengan nilai suku bunga yang diberikan saat objek dibuat.

6. `public void tambahBungaAkhirBulan()`

Merupakan **method** yang digunakan untuk menghitung dan menambahkan bunga ke saldo rekening tabungan pada akhir bulan.

7. `double nominalBunga = saldo * (sukuBunga / 100);`

Menghitung **nominal bunga** berdasarkan saldo rekening dan persentase suku bunga.

8. `saldo += nominalBunga;`

Menambahkan **nominal bunga ke saldo rekening**. `saldo` dapat diakses langsung karena pada class `Rekening` menggunakan modifier `protected`.

9. `riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));`

Menambahkan **transaksi bunga ke dalam riwayat transaksi** rekening.

10. `System.out.println(...)`

Menampilkan **informasi bahwa bunga berhasil ditambahkan** beserta persentase dan nominal bunga.

c) Langkah 3: Membuat Subclass `RekeningGiro`

Buat file baru bernama `RekeningGiro.java`. Produk Giro digunakan pebisnis dan memiliki fitur tambahan yaitu Batas Overdraft (Limit Pinjaman).

```
1 package pekan4;
2
3 public class RekeningGiro extends Rekening {
4     private double batasOverdraft;
5
6     public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
7         // Memanggil inisialisasi dasar dari Superclass
8         super(nomor, nama, saldoAwal, pinAwal);
9         this.batasOverdraft = batasOverdraft;
10    }
11
12    // Getter khusus Giro
13    public double getBatasOverdraft() {
14        return batasOverdraft;
15    }
16 }
```

Berikut penjelasan Langkah-langkah kerjanya:

1. `public class RekeningGiro extends Rekening`
Membuat class `RekeningGiro` sebagai **turunan dari class `Rekening`**, sehingga dapat menggunakan atribut dan method yang diwariskan dari class induknya.
2. `private double batasOverdraft;`
Menyimpan **batas overdraft**, yaitu batas saldo negatif yang diperbolehkan pada rekening giro. `private` berarti atribut ini hanya dapat diakses langsung dari dalam class `RekeningGiro`.
3. `public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft)`
Merupakan **constructor** untuk membuat objek `RekeningGiro` dengan menerima nomor rekening, nama pemilik, saldo awal, PIN, dan batas overdraft.
4. `super(nomor, nama, saldoAwal, pinAwal);`
Memanggil **constructor dari class induk `Rekening`** untuk menginisialisasi data dasar rekening. `super()` wajib berada di baris pertama constructor.
5. `this.batasOverdraft = batasOverdraft;`
Mengisi atribut `batasOverdraft` dengan nilai batas overdraft yang diberikan saat objek dibuat.
6. `public double getBatasOverdraft()`
Merupakan **getter** yang digunakan untuk mengambil atau mendapatkan nilai batas overdraft dari rekening giro.
7. `return batasOverdraft;`
Mengembalikan nilai batas overdraft ketika method `getBatasOverdraft()` dipanggil.

B. Tugas/Latihan Praktikum

Sekarang sistem telah memiliki berbagai bentuk rekening. Silahkan memperbarui antarmuka CLI di `Main.java` agar mendukung produk-produk baru ini secara interaktif.

Tugas 1 : Pembuatan Multi-Produk

Modifikasi case 1: Buka Rekening Baru di Main.java.

1. Setelah user menginputkan Saldo Awal dan PIN, berikan opsi produk:

"Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis"

- Tambahkan kode pada class Main.java tepatnya pada case 1:

```
switch (pilihan) {  
    case 1:  
        System.out.print("Masukkan No Rekening: ");  
        String no = input.nextLine();  
        System.out.print("Masukkan Nama Pemilik: ");  
        String nama = input.nextLine();  
        System.out.print("Masukkan Saldo Awal: ");  
        double saldo = input.nextDouble();  
        input.nextLine();  
        System.out.print("Masukkan PIN (6 digit): ");  
        String pin = input.nextLine();  
  
        Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);  
        daftarRekening.add(rekeningBaru);  
        akunAktif = rekeningBaru;  
        break;
```

Gambar di atas merupakan kode sebelum di ubah

```
switch (pilihan) {  
    case 1:  
        System.out.print("Masukkan No Rekening: ");  
        String no = input.nextLine();  
        System.out.print("Masukkan Nama Pemilik: ");  
        String nama = input.nextLine();  
        System.out.print("Masukkan Saldo Awal: ");  
        double saldo = input.nextDouble();  
        input.nextLine();  
        System.out.print("Masukkan PIN (6 digit): ");  
        String pin = input.nextLine();  
  
        // Pilihan produk  
        System.out.println("Pilih Produk:");  
        System.out.println("1. Tabungan Umum");  
        System.out.println("2. Giro Bisnis");  
        System.out.print("Pilihan: ");  
        int pilihanProduk = input.nextInt();  
        input.nextLine();  
  
        Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);  
        daftarRekening.add(rekeningBaru);  
        akunAktif = rekeningBaru;  
        break;
```

Gambar diatas ini merupakan kode yang sudah ditambahkan, sekarang implementasi opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis" sudah ada.

Kode yang di tambahkan adalah:

```
// Pilihan produk

System.out.println("Pilih Produk:");

System.out.println("1. Tabungan Umum");

System.out.println("2. Giro Bisnis");

System.out.print("Pilihan: ");

int pilihanProduk = scanner.nextInt();

scanner.nextLine();
```

Fungsi kode tersebut:

- `System.out.println("Pilih Produk:");`

Menampilkan tulisan pilihan produk.

- `System.out.println("1. Tabungan Umum");`

Menampilkan pilihan Tabungan Umum.

- `System.out.println("2. Giro Bisnis");`

Menampilkan pilihan Giro Bisnis.

- `int pilihanProduk = scanner.nextInt();`

Menyimpan pilihan user (1 atau 2) ke dalam variabel `pilihanProduk`.

Maka outputnya seperti dibawah ini:

```

=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:
1
Masukkan No Rekening: 12345
Masukkan Nama Pemilik: Gina
Masukkan Saldo Awal: 2000000
Masukkan PIN (6 digit): 654321
Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilihan: 1
Rekening atas nama Gina berhasil dibuat dengan saldo Rp2000000.0

```

2. Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.

- Ganti kode dibawah ini (Kode berada pada class Main.java)

```

Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);
daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;
break;

```

- Menjadi seperti ini

```

Rekening rekeningBaru;

if (pilihanProduk == 1) {
    System.out.print("Masukkan Suku Bunga (%): ");
    double sukuBunga = input.nextDouble();
    input.nextLine();

    rekeningBaru = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
} else {
    rekeningBaru = new Rekening(no, nama, saldo, pin);
}

daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;
break;

```

Berikut penjelasannya:

- 1) Rekening rekeningBaru;

Mendeklarasikan variabel rekeningBaru dengan tipe **Rekening**. Variabel ini nantinya dapat menyimpan objek RekeningTabungan maupun Rekening.

- 2) if (pilihanProduk == 1) {

Mengecek apakah user memilih **produk Tabungan Umum** dengan memasukkan pilihan 1.

3) `System.out.print("Masukkan Suku Bunga (%): ");`

Meminta user memasukkan **suku bunga dalam persen** untuk rekening tabungan.

4) `double sukuBunga = input.nextDouble();`

Menyimpan nilai **suku bunga** yang dimasukkan user ke dalam variabel `sukuBunga`.

5) `rekeningBaru = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);`

Membuat **objek RekeningTabungan** menggunakan nomor rekening, nama, saldo awal, PIN, dan suku bunga.

6) `} else {`

Menjalankan bagian ini jika user **tidak memilih Tabungan**, yaitu ketika pilihan produk bukan 1.

7) `rekeningBaru = new Rekening(no, nama, saldo, pin);`

Membuat **objek Rekening** menggunakan data nomor rekening, nama, saldo awal, dan PIN.

8) `daftarRekening.add(rekeningBaru);`

Menambahkan objek rekening yang telah dibuat ke dalam **daftar rekening**.

9) `akunAktif = rekeningBaru;`

Menjadikan rekening yang baru dibuat sebagai **akun yang sedang aktif**.

10) break;

Menghentikan proses pada case 1 dan kembali ke **menu utama**.

Maka Output program akan seperti gambar dibawah ini:

```
=== SISTEM PERBANKAN MINI ===
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:
1
Masukkan No Rekening: 54321
Masukkan Nama Pemilik: Gina
Masukkan Saldo Awal: 200000
Masukkan PIN (6 digit): 654321
Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilihan: 1
Masukkan Suku Bunga (%): 3
Rekening atas nama Gina berhasil dibuat dengan saldo Rp200000.0
```

3. Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.

- Awalnya kode dibawah ini (Kode berada pada class Main.java)

```
} else {
    rekeningBaru = new Rekening(no, nama, saldo, pin);
```

- Kemudian menjadi seperti ini

```
} else if (pilihanProduk == 2) {
    System.out.print("Masukkan Batas Overdraft: ");
    double batasOverdraft = input.nextDouble();
    input.nextLine();

    rekeningBaru = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);
} else {
    System.out.println("Pilihan produk tidak valid.");
    break;
}
```

Berikut penjelasannya:

1) } else if (pilihanProduk == 2) {

Mengecek apakah user memilih **produk Giro Bisnis** dengan memasukkan pilihan 2.

2) System.out.print("Masukkan Batas Overdraft: ");

Meminta user memasukkan **batas overdraft atau limit pinjaman** yang diberikan pada rekening giro.

3) double batasOverdraft = input.nextDouble();

Menyimpan nilai **batas overdraft** yang dimasukkan user ke dalam variabel batasOverdraft.

4) input.nextLine();

Membersihkan sisa **input Enter** setelah user memasukkan angka agar input berikutnya tidak bermasalah.

5) rekeningBaru = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);

Membuat objek **RekeningGiro** menggunakan nomor rekening, nama, saldo awal, PIN, dan batas overdraft.

6) } else {

Menangani kondisi ketika user memilih selain 1 atau 2.

7) System.out.println("Pilihan produk tidak valid.");

Menampilkan pesan bahwa pilihan produk yang dimasukkan tidak valid

Maka outputnya akan seperti ini:

```
=== SISTEM PERBANKAN MINI ===  
  
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi (Riwayat)  
0. Keluar  
Pilih menu:  
1  
Masukkan No Rekening: 1234  
Masukkan Nama Pemilik: Gina  
Masukkan Saldo Awal: 200000  
Masukkan PIN (6 digit): 123456  
Pilih Produk:  
1. Tabungan Umum  
2. Giro Bisnis  
Pilihan: 2  
Masukkan Batas Overdraft: 500000  
Rekening atas nama Gina berhasil dibuat dengan saldo Rp200000.0
```

4. Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

Pada tugas 4 ini kita Cuma mempertahankan kode program bagian ini:

```
daftarRekening.add(rekeningBaru);  
akunAktif = rekeningBaru;
```

Berikut Penjelasannya:

- 1) daftarRekening.add(rekeningBaru);

Menyimpan objek rekening yang telah dibuat ke dalam daftarRekening.

- 2) akunAktif = rekeningBaru;

Menjadikan rekening yang baru dibuat sebagai akun yang sedang aktif.

Tugas 2: Simulasi Waktu Akhir Bulan

1. Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.

- Tambahkan menu 7 (kode ini di class Main.java pada bagian case 1)

```
while (isRunning) {
    System.out.println("\nMenu Utama:");
    System.out.println("1. Buka Rekening Baru");
    System.out.println("2. Setor Tunai");
    System.out.println("3. Tarik Tunai");
    System.out.println("4. Cek Informasi Rekening");
    System.out.println("5. Ganti Akun");
    System.out.println("6. Cetak Mutasi (Riwayat)");
    System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
    System.out.println("0. Keluar");
    System.out.println("Pilih menu: ");
```

Maka outputnya seperti ini:

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu:
```

2. Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof.

- Tambahkan case 7 di class Main.java

```
case 7:
    if (akunAktif instanceof RekeningTabungan) {
        System.out.println("Akun aktif merupakan Rekening Tabungan.");
    } else {
        System.out.println("Simulasi hanya dapat dilakukan untuk rekening Tabungan.");
    }
    break;
```

Berikut penjelasannya:

1) case 7:

Menjalankan proses ketika user memilih Simulasi Akhir Bulan.

2) `akunAktif instanceof RekeningTabungan`

Memeriksa apakah objek yang sedang tersimpan di `akunAktif` merupakan objek `RekeningTabungan`.

3) `System.out.println("Akun aktif merupakan Rekening Tabungan.");`

Menampilkan pesan jika akun aktif memang merupakan rekening tabungan.

4) `else`

Dijalankan jika akun aktif bukan `RekeningTabungan`, misalnya `RekeningGiro`.

5) `break;`

Menghentikan proses case 7 dan kembali ke menu utama.

Maka outputnya:

```
1
Masukkan No Rekening: 1234
Masukkan Nama Pemilik: Gina
Masukkan Saldo Awal: 200000
Masukkan PIN (6 digit): 123456
Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilihan: 1
Masukkan Suku Bunga (%): 500000
Rekening atas nama Gina berhasil dibuat dengan saldo Rp200000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu:
7
Akun aktif merupakan Rekening Tabungan.
```

3. Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().

- Tambahkan kode dibawah ini pada case 7

```
RekeningTabungan tabungan = (RekeningTabungan)
akunAktif; tabungan.tambahBungaAkhirBulan();
```

Berikut penjelasan:

1) RekeningTabungan tabungan = (RekeningTabungan) akunAktif;

Melakukan downcasting, yaitu mengubah referensi akunAktif dari tipe Rekening menjadi tipe RekeningTabungan.

2) tabungan.tambahBungaAkhirBulan();

Memanggil method tambahBungaAkhirBulan() untuk menghitung dan menambahkan bunga ke saldo tabungan.

4. Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

```
case 7:
    if (akunAktif instanceof RekeningTabungan) {
        RekeningTabungan tabungan = (RekeningTabungan) akunAktif;
        tabungan.tambahBungaAkhirBulan();
    } else {
        System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
    }
    break;
```

Maka untuk output akhirnya akan seperti ini:

```

=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu:
1
Masukkan No Rekening: 1234
Masukkan Nama Pemilik: Gina
Masukkan Saldo Awal: 200.000
Masukkan PIN (6 digit): 123456
Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilihan: 1
Masukkan Suku Bunga (%): 4
Rekening atas nama Gina berhasil dibuat dengan saldo Rp200.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu:
7
Bunga 4.0% berhasil ditambahkan: Rp8.0

```

```

2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu:
1
Masukkan No Rekening: 1234
Masukkan Nama Pemilik: Gina
Masukkan Saldo Awal: 200.000
Masukkan PIN (6 digit): 123456
Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilihan: 2
Masukkan Batas Overdraft: 4
Rekening atas nama Gina berhasil dibuat dengan saldo Rp200.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu:
7
Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.

```

C. Link repository GitHub

https://github.com/GinaRamadhani/PrakPBO_B_2511533014.git

BAB III

PENUTUP

3.1 Kesimpulan

Praktikum Ekspansi Produk Keuangan (Inheritance) menunjukkan bahwa konsep inheritance dapat digunakan untuk mengembangkan berbagai jenis produk rekening dari satu class induk Rekening. Melalui RekeningTabungan dan RekeningGiro, setiap produk dapat memiliki atribut dan fitur khusus tanpa harus membuat class yang sepenuhnya terpisah. Penerapan upcasting, downcasting, dan *instanceof* juga membantu program mengenali jenis rekening dan menjalankan fitur yang sesuai, seperti simulasi bunga yang hanya berlaku pada rekening tabungan.

DAFTAR PUSTAKA

Nurfiah. (2026). *Modul praktikum 4 pemrograman berorientasi objek–Java*.
Fakultas Teknologi Informasi, Universitas Andalas.